PHP Arithmetic Operations & Number Theory Suite

Comprehensive Technical Documentation, Source Code & Execution Guide
Covering Tasks 1 through 6

Language & Runtime	PHP 8.x (Tested on PHP 8.5.10 CLI & Web SAPI)
Script Architecture	Fully Self-Contained Standalone Scripts + Master Runner
Input Flexibility	Dual-Mode (CLI Arguments, Web GET Parameters, Preset Defaults)
Tasks Included	Arithmetic (+,-,*,/,%), Parity, Odd Range, Even Range, Factors, GCF

1. Executive Summary & Architecture

This project implements a complete mathematical and number-analysis suite in PHP. Each task is engineered as an independent, fully functional standalone script that can be executed directly in terminal CLI environments or loaded through a web browser. The scripts are built with defensive programming patterns including zero-division guards, typecasting, input sanitization, and mathematical optimizations such as the Euclidean Algorithm.

File Manifest & Directory Structure

Filename	Task Scope	Primary Methods & Operators
`task1_arithmetic.php`	Task 1: Basic & Advanced Arithmetic	+, -, *, /, %, **, Zero-division guards
`task2_odd_even.php`	Task 2: Odd or Even Parity Evaluation	% 2 parity check, zero handling
`task3_odd_range.php`	Task 3: Odd Numbers Under num1 Range	for loop, modulo filtering, statistics
`task4_even_range.php`	Task 4: Even Numbers Under num2 Range	for loop, modulo filtering, statistics

task5_factors.php	Task 5: Factor Extraction (num1 & num2)	Divisor scanning, array_intersect()
task6_gcf.php	Task 6: Greatest Common Factor (GCF)	Euclidean Algorithm + Factor Intersection
run_all.php	Master Runner: Executes All 6 Tasks	Unified execution with single input pair

2. Execution Guide & Dual-Mode Input Mechanism

Each script incorporates intelligent input resolution, allowing it to dynamically adapt to three operational modes:
1. Preset Defaults: Directly editing $default_num1 and $default_num2 at the top of the file.
2. Command-Line Arguments: Passing custom inputs via terminal parameters ($argv).
3. Web GET Query: Passing query parameters via URL (?num1=...&num2=...) when accessed over HTTP.

```
# 1. Run with default preset numbers:
php task1_arithmetic.php

# 2. Run with custom arguments (num1=24, num2=18):
php task1_arithmetic.php 24 18

# 3. Run all tasks at once through the master runner:
php run_all.php 24 18

# 4. Start local PHP development server for browser testing:
php -S localhost:8000
# Then visit: http://localhost:8000/task1_arithmetic.php?num1=24&num2=18
```

3.1 Task 1: Perform All Arithmetic Operators (+, -, *, /, %)

Computes the fundamental binary arithmetic operations between num1 and num2. Crucially, it guards against division by zero for both standard division (/) and modulus (%), avoiding fatal PHP DivisionByZeroError exceptions.

Source Code: task1_arithmetic.php

```
<?php
/**
```

```
 * Task 1: Perform all arithmetic operators (+, -, *, /, %)
 *
 * Supports CLI arguments ($argv), Web GET params ($_GET), or preset default
values.
 */

// Preset default values
$default_num1 = 24;
$default_num2 = 18;

// Resolve inputs based on execution environment
if (PHP_SAPI === 'cli') {
    $num1 = isset($argv[1]) && is_numeric($argv[1]) ? (float)$argv[1] :
$default_num1;
    $num2 = isset($argv[2]) && is_numeric($argv[2]) ? (float)$argv[2] :
$default_num2;
    $nl = "\n";
} else {
    header('Content-Type: text/html; charset=utf-8');
    $num1 = isset($_GET['num1']) && is_numeric($_GET['num1']) ? (float)
$_GET['num1'] : $default_num1;
    $num2 = isset($_GET['num2']) && is_numeric($_GET['num2']) ? (float)
$_GET['num2'] : $default_num2;
    $nl = "<br>";
    echo "<pre style='font-family: monospace; font-size: 15px;'>";
}

echo "================================================" . $nl;
echo "TASK 1: ARITHMETIC OPERATORS (+, -, *, /, %, **)" . $nl;
echo "================================================" . $nl;
echo "Input num1 : $num1" . $nl;
echo "Input num2 : $num2" . $nl;
echo "------------------------------------------------" . $nl;

// 1. Addition (+)
$sum = $num1 + $num2;
echo "[+] Addition:        $num1 + $num2 = $sum" . $nl;

// 2. Subtraction (-)
$difference = $num1 - $num2;
echo "[-] Subtraction:     $num1 - $num2 = $difference" . $nl;

// 3. Multiplication (*)
$product = $num1 * $num2;
echo "[*] Multiplication:  $num1 * $num2 = $product" . $nl;

// 4. Division (/) with Division by Zero Guard
if ($num2 == 0) {
    echo "[/] Division:        Undefined (Cannot divide by zero)" . $nl;
```

```
} else {
    $quotient = $num1 / $num2;
    $quotient_display = is_float($quotient) ? round($quotient, 4) :
$quotient;
    echo "[/] Division:        $num1 / $num2 = $quotient_display" . $nl;
}

// 5. Modulus (%) with Division by Zero Guard
if ($num2 == 0) {
    echo "[%] Modulus:         Undefined (Cannot divide by zero)" . $nl;
} else {
    $remainder = (int)$num1 % (int)$num2;
    echo "[%] Modulus:         $num1 % $num2 = $remainder" . $nl;
}

// 6. Exponentiation (**)
$power = $num1 ** $num2;
echo "[**] Exponentiation: $num1 ** $num2 = " . (is_infinite($power) ?
"Infinity (overflow)" : $power) . $nl;

echo "================================================" . $nl;

if (PHP_SAPI !== 'cli') {
    echo "</pre>";
}
```

Technical Walkthrough & Key Lines

- Line 8-9: Defines editable default variables ($default_num1 = 24, $default_num2 = 18).
- Line 12-22: Detects runtime environment (CLI vs Web SAPI) and resolves num1 and num2.
- Line 31: Addition ($sum = $num1 + $num2).
- Line 35: Subtraction ($difference = $num1 - $num2).
- Line 39: Multiplication ($product = $num1 * $num2).
- Line 43-49: Division with conditional check ($num2 == 0) to prevent fatal errors.
- Line 52-58: Modulus operation with zero-divisor check.
- Line 61: Exponentiation ($num1 ** $num2) with overflow detection.

Terminal Execution Verification

Command executed: php task1_arithmetic.php 24 18

```
================================================
TASK 1: ARITHMETIC OPERATORS (+, -, *, /, %, **)
================================================
Input num1 : 24
Input num2 : 18
------------------------------------------------
```

```
[+] Addition:        24 + 18 = 42
[-] Subtraction:     24 - 18 = 6
[*] Multiplication:  24 * 18 = 432
[/] Division:        24 / 18 = 1.3333
[%] Modulus:         24 % 18 = 6
[**] Exponentiation: 24 ** 18 = 6.9791470795844E+24
================================================
```

3.2 Task 2: Check Odd or Even

Evaluates the mathematical parity of num1 and num2. An integer n is defined as even if n is divisible by 2 with no remainder (n % 2 === 0). If a remainder of 1 (or -1 for negative numbers) exists, it is classified as odd. Zero is mathematically classified as even.

Source Code: task2_odd_even.php

```php
<?php
/**
 * Task 2: Check Odd or Even
 *
 * Determines whether the given numbers are odd or even using the modulus
operator (%).
 * An integer n is even if n % 2 === 0, otherwise it is odd.
 */

// Preset default values
$default_num1 = 24;
$default_num2 = 18;

// Resolve inputs based on execution environment
if (PHP_SAPI === 'cli') {
    $num1 = isset($argv[1]) && is_numeric($argv[1]) ? (int)$argv[1] :
$default_num1;
    $num2 = isset($argv[2]) && is_numeric($argv[2]) ? (int)$argv[2] :
$default_num2;
    $nl = "\n";
} else {
    header('Content-Type: text/html; charset=utf-8');
    $num1 = isset($_GET['num1']) && is_numeric($_GET['num1']) ? (int)
$_GET['num1'] : $default_num1;
    $num2 = isset($_GET['num2']) && is_numeric($_GET['num2']) ? (int)
$_GET['num2'] : $default_num2;
    $nl = "<br>";
    echo "<pre style='font-family: monospace; font-size: 15px;'>";
}

if (!function_exists('checkOddOrEven')) {
```

```
    /**
     * Helper function to determine parity
     */
    function checkOddOrEven(int $num): string {
        if ($num % 2 === 0) {
            return "$num is EVEN (Reason: $num % 2 === 0, divisible by 2 with
0 remainder)";
        } else {
            return "$num is ODD  (Reason: $num % 2 !== 0, remainder is " .
($num % 2) . ")";
        }
    }
}

echo "================================================" . $nl;
echo "TASK 2: CHECK ODD OR EVEN" . $nl;
echo "================================================" . $nl;
echo "Input num1 : $num1" . $nl;
echo "Input num2 : $num2" . $nl;
echo "------------------------------------------------" . $nl;

echo "[Result for num1]: " . checkOddOrEven($num1) . $nl;
echo "[Result for num2]: " . checkOddOrEven($num2) . $nl;

echo "================================================" . $nl;

if (PHP_SAPI !== 'cli') {
    echo "</pre>";
}
```

Technical Walkthrough & Key Lines

- Line 24-35: Declares helper function checkOddOrEven(int $num) with function_exists guard.
- Line 29: Evaluates ($num % 2 === 0) using strict type comparison.
- Line 30: Returns formatted confirmation of even status and zero remainder.
- Line 32: Returns formatted confirmation of odd status with exact remainder value.
- Line 44-45: Executes parity evaluation for both inputs and prints results.

Terminal Execution Verification

Command executed: php task2_odd_even.php 24 17

```
================================================
TASK 2: CHECK ODD OR EVEN
================================================
Input num1 : 24
Input num2 : 17
```

```
--------------------------------------------------
[Result for num1]: 24 is EVEN (Reason: 24 % 2 === 0, divisible by 2 with 0
remainder)
[Result for num2]: 17 is ODD  (Reason: 17 % 2 !== 0, remainder is 1)
==================================================
```

3.3 Task 3: Display All Odd Numbers Under the Range of num1

Iterates from 1 up to the upper boundary specified by num1, identifying and accumulating all odd integers ($i % 2 !== 0). It also aggregates statistical metrics: total count, smallest odd, largest odd, and cumulative sum.

Source Code: task3_odd_range.php

```
<?php
/**
 * Task 3: Display all odd numbers under the range of num1
 *
 * Iterates through the range [1 .. num1] and filters odd numbers (i % 2 !==
0).
 */

// Preset default values
$default_num1 = 24;
$default_num2 = 18;

// Resolve inputs based on execution environment
if (PHP_SAPI === 'cli') {
    $num1 = isset($argv[1]) && is_numeric($argv[1]) ? (int)$argv[1] :
$default_num1;
    $nl = "\n";
} else {
    header('Content-Type: text/html; charset=utf-8');
    $num1 = isset($_GET['num1']) && is_numeric($_GET['num1']) ? (int)
$_GET['num1'] : $default_num1;
    $nl = "<br>";
    echo "<pre style='font-family: monospace; font-size: 15px;'>";
}

echo "==================================================" . $nl;
echo "TASK 3: ALL ODD NUMBERS UNDER THE RANGE OF num1" . $nl;
echo "==================================================" . $nl;
echo "Upper Bound (num1) : $num1" . $nl;
echo "Range Evaluated    : 1 to $num1" . $nl;
echo "--------------------------------------------------" . $nl;
```

```
if ($num1 < 1) {
    echo "Notice: num1 is less than 1. No positive odd numbers exist in range
[1 .. $num1]." . $nl;
} else {
    $oddNumbers = [];
    for ($i = 1; $i <= $num1; $i++) {
        if ($i % 2 !== 0) {
            $oddNumbers[] = $i;
        }
    }

    echo "Odd numbers found (" . count($oddNumbers) . " total):" . $nl;
    echo implode(", ", $oddNumbers) . $nl . $nl;
    echo "Summary Statistics:" . $nl;
    echo "- Smallest odd in range: " . min($oddNumbers) . $nl;
    echo "- Largest odd in range : " . max($oddNumbers) . $nl;
    echo "- Sum of odd numbers   : " . array_sum($oddNumbers) . $nl;
}

echo "================================================" . $nl;

if (PHP_SAPI !== 'cli') {
    echo "</pre>";
}
```

Technical Walkthrough & Key Lines

- Line 28-30: Boundary validation to ensure num1 >= 1.
- Line 31: Initializes accumulator array $oddNumbers = [].
- Line 32-36: Standard for loop iterating from $i = 1 to $num1, filtering odd values.
- Line 38-44: Displays formatted list using implode(', ', $oddNumbers) alongside statistics (min, max, count, array_sum).

Terminal Execution Verification

Command executed: php task3_odd_range.php 15

```
================================================
TASK 3: ALL ODD NUMBERS UNDER THE RANGE OF num1
================================================
Upper Bound (num1) : 15
Range Evaluated    : 1 to 15
------------------------------------------------
Odd numbers found (8 total):
1, 3, 5, 7, 9, 11, 13, 15

Summary Statistics:
- Smallest odd in range: 1
```

```
- Largest odd in range : 15
- Sum of odd numbers   : 64
================================================
```

3.4 Task 4: Display All Even Numbers Under the Range of num2

Iterates from 1 up to the upper boundary specified by num2, filtering all positive even integers ($i % 2 === 0). It compiles summary metrics including total count, minimum even value, maximum even value, and arithmetic sum of the series.

Source Code: task4_even_range.php

```php
<?php
/**
 * Task 4: Display all even numbers under the range of num2
 *
 * Iterates through the range [1 .. num2] and filters even numbers (i % 2 ===
0).
 */

// Preset default values
$default_num1 = 24;
$default_num2 = 18;

// Resolve inputs based on execution environment
if (PHP_SAPI === 'cli') {
    // In CLI, if 2 args passed, num2 is $argv[2]; if only 1 arg passed,
treat as num2 or num1
    $num2 = isset($argv[2]) && is_numeric($argv[2]) ? (int)$argv[2] :
(isset($argv[1]) && is_numeric($argv[1]) ? (int)$argv[1] : $default_num2);
    $nl = "\n";
} else {
    header('Content-Type: text/html; charset=utf-8');
    $num2 = isset($_GET['num2']) && is_numeric($_GET['num2']) ? (int)
$_GET['num2'] : $default_num2;
    $nl = "<br>";
    echo "<pre style='font-family: monospace; font-size: 15px;'>";
}

echo "================================================" . $nl;
echo "TASK 4: ALL EVEN NUMBERS UNDER THE RANGE OF num2" . $nl;
echo "================================================" . $nl;
echo "Upper Bound (num2) : $num2" . $nl;
echo "Range Evaluated    : 1 to $num2" . $nl;
echo "------------------------------------------------" . $nl;
```

```
if ($num2 < 2) {
    echo "Notice: num2 is less than 2. No positive even numbers exist in
range [1 .. $num2]." . $nl;
} else {
    $evenNumbers = [];
    for ($i = 1; $i <= $num2; $i++) {
        if ($i % 2 === 0) {
            $evenNumbers[] = $i;
        }
    }

    echo "Even numbers found (" . count($evenNumbers) . " total):" . $nl;
    echo implode(", ", $evenNumbers) . $nl . $nl;
    echo "Summary Statistics:" . $nl;
    echo "- Smallest even in range: " . min($evenNumbers) . $nl;
    echo "- Largest even in range : " . max($evenNumbers) . $nl;
    echo "- Sum of even numbers   : " . array_sum($evenNumbers) . $nl;
}

echo "================================================" . $nl;

if (PHP_SAPI !== 'cli') {
    echo "</pre>";
}
```

Technical Walkthrough & Key Lines

- Line 28-30: Verifies upper boundary num2 >= 2 for positive even numbers.
- Line 31: Initializes accumulator array $evenNumbers = [].
- Line 32-36: Loop scanning integers from 1 up to num2, selecting those where ($i % 2 === 0).
- Line 38-44: Outputs comma-separated numbers and statistical summary (min, max, count, array_sum).

Terminal Execution Verification

Command executed: php task4_even_range.php 24 18

```
================================================
TASK 4: ALL EVEN NUMBERS UNDER THE RANGE OF num2
================================================
Upper Bound (num2) : 18
Range Evaluated    : 1 to 18
------------------------------------------------
Even numbers found (9 total):
2, 4, 6, 8, 10, 12, 14, 16, 18

Summary Statistics:
- Smallest even in range: 2
```

```
- Largest even in range : 18
- Sum of even numbers   : 90
================================================
```

3.5 Task 5: Display All Factors of num1 and num2

Calculates all positive integer divisors (factors) for both num1 and num2. An integer d is a factor of N if N % d === 0. The script calculates factors for num1, factors for num2, and then performs an array intersection (array_intersect) to display the set of shared common factors.

Source Code: task5_factors.php

```
<?php
/**
 * Task 5: Display all factors of num1 and num2
 *
 * A factor (divisor) of an integer N is an integer d such that N % d === 0.
 * This script extracts all positive factors for both num1 and num2,
 * and also highlights the common factors shared between them.
 */

// Preset default values
$default_num1 = 24;
$default_num2 = 18;

// Resolve inputs based on execution environment
if (PHP_SAPI === 'cli') {
    $num1 = isset($argv[1]) && is_numeric($argv[1]) ? abs((int)$argv[1]) :
$default_num1;
    $num2 = isset($argv[2]) && is_numeric($argv[2]) ? abs((int)$argv[2]) :
$default_num2;
    $nl = "\n";
} else {
    header('Content-Type: text/html; charset=utf-8');
    $num1 = isset($_GET['num1']) && is_numeric($_GET['num1']) ? abs((int)
$_GET['num1']) : $default_num1;
    $num2 = isset($_GET['num2']) && is_numeric($_GET['num2']) ? abs((int)
$_GET['num2']) : $default_num2;
    $nl = "<br>";
    echo "<pre style='font-family: monospace; font-size: 15px;'>";
}

if (!function_exists('getFactors')) {
    /**
     * Returns an array of positive factors for a given integer.
     */
```

```
    function getFactors(int $n): array {
        if ($n === 0) {
            return []; // 0 has infinitely many non-zero factors
        }
        $factors = [];
        for ($i = 1; $i <= $n; $i++) {
            if ($n % $i === 0) {
                $factors[] = $i;
            }
        }
        return $factors;
    }
}

echo "==================================================" . $nl;
echo "TASK 5: FACTORS OF num1 AND num2" . $nl;
echo "==================================================" . $nl;
echo "Input num1 : $num1" . $nl;
echo "Input num2 : $num2" . $nl;
echo "--------------------------------------------------" . $nl;

$factors1 = getFactors($num1);
$factors2 = getFactors($num2);
$commonFactors = array_values(array_intersect($factors1, $factors2));

// Display factors of num1
echo "[+] Factors of num1 ($num1):" . $nl;
if (empty($factors1)) {
    echo "    (Every non-zero integer divides 0)" . $nl;
} else {
    echo "    " . implode(", ", $factors1) . " (Count: " . count($factors1) .
")" . $nl;
}
echo $nl;

// Display factors of num2
echo "[+] Factors of num2 ($num2):" . $nl;
if (empty($factors2)) {
    echo "    (Every non-zero integer divides 0)" . $nl;
} else {
    echo "    " . implode(", ", $factors2) . " (Count: " . count($factors2) .
")" . $nl;
}
echo $nl;

// Display Common Factors
echo "[*] Common Factors of $num1 and $num2:" . $nl;
if (empty($commonFactors)) {
    echo "    None found." . $nl;
```

```
} else {
    echo "    " . implode(", ", $commonFactors) . " (Count: " .
count($commonFactors) . ")" . $nl;
}

echo "================================================" . $nl;

if (PHP_SAPI !== 'cli') {
    echo "</pre>";
}
```

Technical Walkthrough & Key Lines

- Line 25-42: Declares getFactors(int $n) function which loops from 1 to $n checking for zero remainder.
- Line 51-52: Extracts factors for both inputs into $factors1 and $factors2.
- Line 53: Computes common factors using array_intersect($factors1, $factors2).
- Line 56-74: Outputs factor list and total count for num1, num2, and the shared subset.

Terminal Execution Verification

Command executed: php task5_factors.php 24 18

```
================================================
TASK 5: FACTORS OF num1 AND num2
================================================
Input num1 : 24
Input num2 : 18
------------------------------------------------
[+] Factors of num1 (24):
    1, 2, 3, 4, 6, 8, 12, 24 (Count: 8)

[+] Factors of num2 (18):
    1, 2, 3, 6, 9, 18 (Count: 6)

[*] Common Factors of 24 and 18:
    1, 2, 3, 6 (Count: 4)
================================================
```

3.6 Task 6: Display Greatest Common Factor (GCF / GCD)

Determines the Greatest Common Factor (GCF), also known as the Greatest Common Divisor (GCD), between num1 and num2. The script provides two distinct computational approaches: 1. Factor Intersection: Selecting the maximum value from the shared factor set computed in Task 5. 2. Euclidean Algorithm: The classic, highly efficient division-with-remainder algorithm

O(log(min(a,b))). It displays every intermediate division step to demonstrate how the final result is reached.

Source Code: task6_gcf.php

```
<?php
/**
 * Task 6: Display Greatest Common Factor (GCF / GCD) of num1 and num2
 *
 * Demonstrates both:
 * 1. Factor Intersection Method (matching common factors from Task 5)
 * 2. Euclidean Algorithm (efficient standard algorithm) with step-by-step
trace
 */

// Preset default values
$default_num1 = 24;
$default_num2 = 18;

// Resolve inputs based on execution environment
if (PHP_SAPI === 'cli') {
    $num1 = isset($argv[1]) && is_numeric($argv[1]) ? abs((int)$argv[1]) :
$default_num1;
    $num2 = isset($argv[2]) && is_numeric($argv[2]) ? abs((int)$argv[2]) :
$default_num2;
    $nl = "\n";
} else {
    header('Content-Type: text/html; charset=utf-8');
    $num1 = isset($_GET['num1']) && is_numeric($_GET['num1']) ? abs((int)
$_GET['num1']) : $default_num1;
    $num2 = isset($_GET['num2']) && is_numeric($_GET['num2']) ? abs((int)
$_GET['num2']) : $default_num2;
    $nl = "<br>";
    echo "<pre style='font-family: monospace; font-size: 15px;'>";
}

if (!function_exists('getFactors')) {
    /**
     * Returns an array of positive factors for an integer.
     */
    function getFactors(int $n): array {
        if ($n === 0) return [];
        $factors = [];
        for ($i = 1; $i <= $n; $i++) {
            if ($n % $i === 0) {
                $factors[] = $i;
            }
        }
        return $factors;
```

```
    }
}

echo "==================================================" . $nl;
echo "TASK 6: GREATEST COMMON FACTOR (GCF / GCD)" . $nl;
echo "==================================================" . $nl;
echo "Input num1 : $num1" . $nl;
echo "Input num2 : $num2" . $nl;
echo "--------------------------------------------------" . $nl;

// Method 1: Factor Intersection
$factors1 = getFactors($num1);
$factors2 = getFactors($num2);
$commonFactors = array_values(array_intersect($factors1, $factors2));

echo "[Method 1: Factor Listing]" . $nl;
echo "Factors of $num1 : " . (!empty($factors1) ? implode(", ", $factors1) :
"N/A") . $nl;
echo "Factors of $num2 : " . (!empty($factors2) ? implode(", ", $factors2) :
"N/A") . $nl;
echo "Common Factors  : " . (!empty($commonFactors) ? implode(", ",
$commonFactors) : "None") . $nl;
$gcfMethod1 = !empty($commonFactors) ? max($commonFactors) : ($num1 ?:
$num2);
echo "Greatest from list: $gcfMethod1" . $nl;
echo "--------------------------------------------------" . $nl;

// Method 2: Euclidean Algorithm (Iterative with step trace)
echo "[Method 2: Euclidean Algorithm Steps]" . $nl;
$a = $num1;
$b = $num2;
$step = 1;

if ($b === 0) {
    $gcf = $a;
    echo "Since num2 is 0, GCF is $a." . $nl;
} else {
    while ($b !== 0) {
        $remainder = $a % $b;
        $quotient = intdiv($a, $b);
        echo "Step $step: $a = ($b * $quotient) + $remainder" . $nl;
        $a = $b;
        $b = $remainder;
        $step++;
    }
    $gcf = $a;
}

echo "--------------------------------------------------" . $nl;
```

```
echo ">> FINAL RESULT: GCF($num1, $num2) = $gcf <<" . $nl;

if ($gcf === 1) {
    echo "Note: The GCF is 1, which means $num1 and $num2 are COPRIME
(relatively prime)." . $nl;
} else {
    echo "Note: $num1 and $num2 share $gcf as their largest common
divisor." . $nl;
}

echo "==================================================" . $nl;

if (PHP_SAPI !== 'cli') {
    echo "</pre>";
}
```

Technical Walkthrough & Key Lines

- Line 43-51: Method 1 computes factors of both numbers and picks max($commonFactors).
- Line 54-73: Method 2 executes the Euclidean algorithm: while ($b !== 0) { remainder = $a % $b; $a = $b; $b = remainder; }.
- Line 63-66: Prints step-by-step arithmetic: $a = ($b * quotient) + remainder.
- Line 76-83: Outputs final GCF and notes whether the numbers are coprime (GCF == 1).

Terminal Execution Verification

Command executed: php task6_gcf.php 24 18

```
==================================================
TASK 6: GREATEST COMMON FACTOR (GCF / GCD)
==================================================
Input num1 : 24
Input num2 : 18
--------------------------------------------------
[Method 1: Factor Listing]
Factors of 24 : 1, 2, 3, 4, 6, 8, 12, 24
Factors of 18 : 1, 2, 3, 6, 9, 18
Common Factors  : 1, 2, 3, 6
Greatest from list: 6
--------------------------------------------------
[Method 2: Euclidean Algorithm Steps]
Step 1: 24 = (18 * 1) + 6
Step 2: 18 = (6 * 3) + 0
--------------------------------------------------
>> FINAL RESULT: GCF(24, 18) = 6 <<
Note: 24 and 18 share 6 as their largest common divisor.
==================================================
```

4. Master Runner: run_all.php

To facilitate instant testing and evaluation of all 6 tasks in a single command, the run_all.php script acts as an orchestration runner. It passes the inputs to each task script and aggregates the complete outputs into a unified report.

```
<?php
/**
 * Master Runner: Executes Tasks 1 through 6 sequentially.
 *
 * Usage in Terminal:
 *   php run_all.php
 *   php run_all.php 24 18
 *   php run_all.php 50 20
 *
 * Usage in Browser:
 *   run_all.php?num1=24&num2=18
 */

$num1 = isset($argv[1]) && is_numeric($argv[1]) ? $argv[1] :
($_GET['num1'] ?? 24);
$num2 = isset($argv[2]) && is_numeric($argv[2]) ? $argv[2] :
($_GET['num2'] ?? 18);

$tasks = [
    'task1_arithmetic.php',
    'task2_odd_even.php',
    'task3_odd_range.php',
    'task4_even_range.php',
    'task5_factors.php',
    'task6_gcf.php'
];

if (PHP_SAPI !== 'cli') {
    echo "<!DOCTYPE html><html><head><title>PHP Tasks 1-6 Execution</title>";
    echo "<style>body{font-family: Arial, sans-serif; background: #f4f6f9;
padding: 20px;}";
    echo ".card{background: #fff; border-radius: 8px; box-shadow: 0 2px 8px
rgba(0,0,0,0.1); margin-bottom: 20px; padding: 15px 20px;}";
    echo "pre{background: #272822; color: #f8f8f2; padding: 15px; border-
radius: 6px; overflow-x: auto;}";
    echo "h1{color: #2c3e50;} h2{color: #34495e; border-bottom: 2px solid
#ecf0f1; padding-bottom: 8px;}</style></head><body>";
    echo "<h1>PHP Assignment Tasks 1 - 6</h1>";
    echo "<p>Currently evaluating: <strong>num1 = $num1</strong>,
<strong>num2 = $num2</strong></p>";
}

foreach ($tasks as $index => $script) {
```

```
    if (PHP_SAPI !== 'cli') {
        echo "<div class='card'>";
    }

    // Simulate CLI arguments for the included script
    $argv = [__FILE__, $num1, $num2];
    $_GET['num1'] = $num1;
    $_GET['num2'] = $num2;

    include $script;

    if (PHP_SAPI !== 'cli') {
        echo "</div>";
    } else {
        echo "\n\n";
    }
}

if (PHP_SAPI !== 'cli') {
    echo "</body></html>";
}
```

5. Algorithmic Complexity & Performance Analysis

Task	Algorithm Used	Time Complexity	Space Complexity
Task 1: Arithmetic	Direct CPU ALU operations	O(1) Constant	O(1) Constant
Task 2: Odd/Even	Modulo parity check (% 2)	O(1) Constant	O(1) Constant
Task 3: Odd Range	Linear iteration with modulo filter	O(num1)	O(num1 / 2)
Task 4: Even Range	Linear iteration with modulo filter	O(num2)	O(num2 / 2)
Task 5: Factors	Exhaustive divisor test (1..N)	O(num1 + num2)	O(Factors count)
Task 6: GCF	Euclidean Algorithm (Division remainder)	O(log(min(a,b)))	O(1) Iterative

6. Edge Cases & Defensive Programming Matrix

Edge Case Scenario	Potential Vulnerability / Error	Mitigation Implemented in Code
num2 = 0 in Task 1	PHP DivisionByZeroError throws fatal uncaught exception	Pre-check ($num2 == 0) returning 'Undefined' before division or modulus.
Non-numeric CLI inputs	Type mismatch or PHP TypeError in arithmetic	is_numeric() validation with safe fallback to default values.
Negative integers in ranges	Infinite loop or incorrect range bounds in Tasks 3 & 4	Validation checks ensuring upper boundary is at least 1 (or 2 for even numbers).
Coprime numbers in Task 6	Potential failure to detect common factors	Correctly concludes GCF = 1 and explicitly flags numbers as coprime.